UNIP

CIÊNCIAS DA COMPUTAÇÃO

GABRIEL VIEIRA DE MEDEIROS – R839IJ4

HIAGO MULLER DE MORAIS – R857BH5

JOÃO PEDRO PEREIRA FRONE – R6591B2

MARCELO FELIPE DOS SANTOS AGANTE – F364313

APS - 2º SEMESTRE – ESTRUTURA DE DADOS

ALGORITMOS DE ORDENAÇÃO

SÃO PAULO

2026

SUMÁRIO

OBJETIVO **4**
INTRODUÇÃO 5
FUNDAMENTAÇÃO TEÓRICA 6
Insertion Sort (Ordenação por Inserção) 9
Selection Sort (Ordenação por Seleção) 11
Merge Sort (Ordenação por Intercalação) 13
Quick Sort (Ordenação Rápida) 16
Tabela Comparativa Estruturada 19
METODOLOGIA 20
Arquitetura do Sistema e Tecnologias 20
Modelagem e Preparação dos Dados 20
Implementação dos Algoritmos de Ordenação e Instrumentação de Métricas 21
Design da Interface Web e Dashboard de Resultados 22
PROJETO DO PROGRAMA 23
Arquitetura de comunicação entre frontend e backend 23
Processamento de ordenação e previsão 23
Demonstração do uso de algoritmos de ordenação 26
Testes, validações e iterações (Benchmarks) 26
Ambiente e procedimentos de execução dos testes 29
Análise Comparativa: Selection Sort x Insertion Sort 30
Análise Comparativa: Merge Sort x Quick Sort 31
Comparativo Geral Consolidado 32
Testes dos Endpoints da API (ApiController) 34
CONCLUSÃO **38**
BIBLIOGRAFIA **39**
ANEXOS **40**
Código GitHub: 40
Youtube: 40
Fichas Técnicas: 40

OBJETIVO

Desenvolver um sistema para análise de desempenho de algoritmos de ordenação aplicado a dados de focos de queimadas detectados por satélite e disponibilizados pelo Instituto Nacional de Pesquisas Espaciais (INPE), referentes aos anos de 2023 e 2024 e abrangendo todo o território nacional, de modo a comparar a eficiência dos algoritmos Insertion Sort, Selection Sort, Merge Sort e Quick Sort no processamento de grandes volumes de dados geográficos.

Para alcançar o objetivo, o trabalho se propõe a:

- importar e tratar os arquivos CSV do INPE, lidando com registros incompletos ou inconsistentes;
- implementar em linguagem Java os quatro algoritmos de ordenação selecionados;
- permitir a exibição dos dados de um estado, selecionado no mapa, ordenados por data, bioma e município, de forma isolada ou combinada;
- registrar e exibir, a cada solicitação, o número de operações realizadas (comparações e substituições) e o tempo de execução de cada algoritmo;
- disponibilizar um painel web interativo que apresente de forma clara os resultados comparativos;
- relacionar os resultados observados com a complexidade teórica de cada algoritmo, expressa na notação Big-O;

As queimadas afetam a biodiversidade, a qualidade do ar e o clima, e o monitoramento contínuo dos biomas brasileiros por meio de imagens de satélite é fundamental para a fiscalização e o combate a atividades ilegais. Esse monitoramento gera bases de dados volumosas, cuja análise depende de algoritmos capazes de organizar e consultar as informações de maneira rápida e confiável.

Nesse contexto, a escolha do algoritmo de ordenação deixa de ser apenas uma questão teórica: o comportamento de cada método diante de milhares ou milhões de registros pode determinar se uma consulta é concluída em segundos ou se torna inviável na prática. Comparar, em dados reais, algoritmos de complexidades distintas permite observar de forma concreta o impacto da escolha da estratégia de ordenação sobre o desempenho.

Além disso, o trabalho integra conhecimentos de Estrutura de Dados, programação orientada a objetos, desenvolvimento web e análise de dados ambientais, aproximando conteúdos estudados em sala de aula de um problema socioambiental relevante.

INTRODUÇÃO

As queimadas representam um problema ambiental que pode afetar a biodiversidade e o equilíbrio dos ecossistemas. O Brasil possui seis biomas continentais, cada um com características ambientais e diversidade biológica próprias (IBGE, 2026). Nesse contexto, o monitoramento dos focos de queimadas por meio de imagens de satélite, realizado pelo Instituto Nacional de Pesquisas Espaciais, fornece informações importantes para a análise da ocorrência e distribuição espacial desses eventos (INPE, 2026).

A ordenação de dados é uma operação fundamental na computação, pois permite organizar conjuntos de informações e facilitar operações posteriores de busca, análise e processamento (Knuth, 1998).

Além da comparação entre os algoritmos de ordenação, o sistema contempla uma funcionalidade complementar de previsão de queimadas para períodos posteriores, utilizando registros históricos organizados cronologicamente. Essa funcionalidade aplica cálculos matemáticos sobre os dados disponíveis para estimar o comportamento dos focos de queimadas no período seguinte, demonstrando também a utilidade da organização temporal das informações no processamento e na análise de dados.

O presente trabalho tem como foco aplicar os conceitos de Estrutura de Dados na prática, desenvolvendo uma aplicação web interativa que consome os dados de queimadas do INPE dos anos de 2023 e 2024. Através da implementação dos algoritmos clássicos *Insertion Sort*, *Selection Sort*, *Merge Sort* e *Quick Sort*, o projeto propõe uma análise empírica comparativa. O intuito é demonstrar como o crescimento da base de dados impacta o tempo de resposta de uma API RESTful, estabelecendo um paralelo direto entre a complexidade teórica (notação Big-O) e o comportamento do software em um cenário real de uso.

FUNDAMENTAÇÃO TEÓRICA

Um algoritmo de ordenação é uma sequência finita de instruções lógicas e matemáticas projetada para reorganizar os elementos de uma coleção (geralmente arranjos ou listas) em uma ordem predeterminada, definida por uma relação de ordem total (ex.: ordem numérico-crescente <= ou alfabética).

A análise de complexidade permite avaliar o crescimento do custo computacional de um algoritmo conforme o aumento da quantidade de elementos processados. Essa análise é importante para compreender o comportamento dos algoritmos em diferentes cenários de entrada (Cormen et al., 2012).

Formalmente, dada uma sequência de N elementos $A = \{a_1, a_2, ..., a_N\}$, o objetivo do algoritmo é produzir uma permutação $A' = \{a'_1, a'_2, ..., a'_N\}$ de forma que:

$a'_1 <= a'_2 <= ... <= a'_N$

Os algoritmos de ordenação classificam-se quanto a dois critérios operacionais primordiais:

- Estabilidade: Um algoritmo é considerado estável se preserva a ordem relativa original de elementos que possuem chaves de ordenação equivalentes.
- Espaço em Memória (*In-place* vs. *Out-of-place*): Um algoritmo é considerado *in-place* quando requer apenas uma quantidade constante de memória auxiliar além do vetor de entrada, ou seja, possui complexidade de espaço O(1).

A necessidade de ordenar dados é anterior à invenção dos computadores eletrônicos. No final do século XIX, Herman Hollerith desenvolveu a máquina tabuladora de cartões perfurados para acelerar o Censo de 1890 dos Estados Unidos. A ordenação física desses cartões através de métodos mecânicos (precursores do *Radix Sort*) mostrou que organizar a informação era pré-requisito para seu processamento.

Com o advento dos primeiros computadores eletrônicos programáveis durante a Década de 1940 (como o ENIAC e o EDVAC), o cenário de processamento era caracterizado por:

1. Memória extremamente escassa e cara: Unidades de memória RAM eram medidas em centenas de *bytes* ou poucas palavras de memória (ex.: tubos de raios catódicos de Williams ou linhas de retardo de mercúrio).
2. Mídias sequenciais lentas: As principais formas de armazenamento persistente eram fitas magnéticas e cartões perfurados.
3. Processamento em Lote (*Batch Processing*): Operações militares, censitárias e contábeis precisavam processar e cruzar registros sem acesso aleatório rápido aos dados.

Os algoritmos de ordenação foram criados para contornar o gargalo da velocidade de busca e do cruzamento de dados. Antes do processamento eletrônico, localizar um registro específico em uma fita magnética desordenada exigia varrer a fita do início ao fim (busca linear O(N)). Se a fita estivesse ordenada, a busca poderia utilizar abordagens como a busca binária, reduzindo drasticamente o tempo de localização para O(log N).

A ordenação viabilizou:

- A consolidação de relatórios financeiros e comerciais.
- A junção de conjuntos de dados (*data merging*).
- Cálculos científicos do Projeto Manhattan e simulações balísticas durante a Segunda Guerra Mundial.

Apesar dos avanços em hardware, a ordenação permanece como uma das operações mais requisitadas na ciência da computação. O surgimento dos Sistemas de Gerenciamento de Bancos de Dados Relacionais (SGBDs) transformou a ordenação em um componente essencial de otimização de consultas.

Nos bancos de dados modernos, algoritmos de ordenação operam principalmente em:

- Cláusulas SQL Explicitadas (ORDER BY e GROUP BY): A ordenação explícita de registros resultantes.
- Operações de Junção via *Sort-Merge Join*: Quando duas tabelas grandes precisam ser conectadas através de uma chave estrangeira, o otimizador do SGBD frequentemente ordena ambas as tabelas e realiza uma intercalação linear, apresentando menor custo que varreduras aninhadas repetidas (*Nested Loops*).
- Eliminação de Duplicatas (DISTINCT): Para agrupar ou remover linhas idênticas, a forma mais eficiente é ordenar os dados primeiro, de forma que registros duplicados fiquem juntos em memória.
- Ordenação Externa (*External Sorting*): Quando o volume de dados ultrapassa a capacidade de memória RAM disponível no servidor, os SGBDs empregam variações do External Merge Sort, dividindo os dados no disco em pequenos blocos (*runs*), ordenando cada bloco na memória e intercalando os blocos gravados no disco através de buffers.

Algoritmos de ordenação

Algoritmos Simples (Complexidade: $O(n^2)$):

- Insertion Sort
- Selection Sort

Divisão e Conquista (Complexidade: O(n log n)):

- Merge Sort

- Quick Sort

Insertion Sort (Ordenação por Inserção)

Descrição do Funcionamento

O Insertion Sort constrói o arranjo final ordenado um elemento por vez. Seu funcionamento é análogo à maneira como as pessoas organizam cartas de baralho nas mãos: assume-se que o primeiro elemento forma uma sublista já ordenada. Em seguida, pega-se o próximo elemento desordenado (denominado chave) e percorre-se a sublista ordenada da direita para a esquerda, deslocando os elementos maiores para uma posição à frente até encontrar a posição correta para inserir a chave.

O Insertion Sort organiza os elementos de maneira incremental, inserindo cada novo elemento em sua posição adequada dentro da parte já ordenada do vetor. Seu desempenho depende significativamente da disposição inicial dos elementos (Knuth, 1998).

Pseudocódigo

```
Algoritmo InsertionSort(A, N)
    Para i de 1 até N - 1 Faça
        chave = A[i]
        j = i - 1

        # Mover os elementos de A[0..i-1] que são maiores que a chave para uma posição à frente de sua posição atual
        Enquanto j >= 0 e A[j] > chave Faça
          A[j + 1] = A[j]
          j = j - 1
        Fim-Enquanto

        A[j + 1] = chave
    Fim-Para
Fim-Algoritmo
```

Complexidade de Tempo e Espaço

- Melhor Caso —O(N): Ocorre quando o arranjo já está totalmente ordenado. O laço interno (Enquanto) executa apenas uma comparação por passagem e nenhuma movimentação.
- Caso Médio — $O(N^2)$: Ocorre para distribuições aleatórias de dados, exigindo em média $N^2 / 4$ comparações e $N^2 / 4$ escritas.
- Pior Caso — $O(N^2)$: Ocorre quando o arranjo está em ordem exatamente inversa. O laço interno desloca todos os elementos do prefixo já processado.
- Complexidade de Espaço: O(1) — Algoritmo estritamente *in-place*.

Estabilidade

Estável - Elementos com chaves iguais não trocam de posição entre si, pois a condição de parada do laço interno trata apenas de elementos estritamente maiores (A[j] > chave).

Aplicação Prática

O Insertion Sort é utilizado na prática como algoritmo de base para conjuntos de dados reduzidos (geralmente N < 16 ou N < 32) dentro de algoritmos híbridos modernos como o Timsort (padrão em Python e Java) e o Introsort (padrão na C++ STL), devido ao seu baixo overhead e excelente aproveitamento de localidade de cache (*cache locality*).

Selection Sort (Ordenação por Seleção)

Descrição do Funcionamento

O Selection Sort divide a lista de entrada em duas partes: uma sublista de itens já ordenados (no início) e uma sublista de itens restantes a serem ordenados. Em cada iteração, o algoritmo busca o menor (ou maior) elemento da sublista não ordenada, realiza uma única troca para colocá-lo no final da sublista ordenada e avança a fronteira em uma posição.

Pseudocódigo

```
Algoritmo SelectionSort(A, N)
   Para i de 0 até N - 2 Faça
      indice_min = i

      # Encontrar o menor elemento na sublista não ordenada
      Para j de i + 1 até N - 1 Faça
        Se A[j] < A[indice_min] Então
              indice_min = j
        Fim-Se
      Fim-Para

      # Trocar o menor elemento encontrado com o primeiro elemento não ordenado
      Se indice_min != i Então
        trocar(A[i], A[indice_min])
      Fim-Se
   Fim-Para
Fim-Algoritmo
```

Complexidade de Tempo e Espaço

- Melhor Caso — $O(N^2)$: Mesmo que o vetor esteja totalmente ordenado, o algoritmo é obrigado a realizar todas as comparações para garantir que encontrou o elemento mínimo de cada iteração.
- Caso Médio — $O(N^2)$: O número total de comparações é sempre fixado em: $(N(N-1)/2)$
- Pior Caso — $O(N^2)$: idêntico ao melhor e médio caso quanto ao número de comparações.
- Complexidade de Espaço: O(1) — Algoritmo *in-place*.

Estabilidade

Instável na sua implementação tradicional baseada em trocas. Uma troca de longa distância pode alterar a ordem relativa de elementos iguais situados entre a posição de destino e o elemento selecionado.

Aplicação Prática

O Selection Sort é preferido em sistemas embarcados onde o custo de gravação na memória (como memória Flash ou EEPROM) é significativamente mais alto do que o custo de leitura, uma vez que ele garante no máximo N - 1 operações de escrita/troca.

Merge Sort (Ordenação por Intercalação)

Descrição do Funcionamento

Proposto por John von Neumann em 1945 (Knuth, 1998), o Merge Sort é o exemplo clássico do paradigma de Divisão e Conquista (*Divide and Conquer*). O algoritmo opera recursivamente em três etapas:

1. Divisão: Divide o vetor de N elementos ao meio, formando dois subvetores de tamanho N/2.
2. Conquista: Ordena recursivamente os dois subvetores chamando a si próprio.
3. Combinação (Merge): Intercala os dois subvetores já ordenados para produzir um único vetor totalmente ordenado.

O Merge Sort utiliza a estratégia de divisão e conquista, dividindo o conjunto de elementos em partes menores, ordenando-as recursivamente e posteriormente combinando os resultados (Cormen et al., 2012).

Pseudocódigo

```
Algoritmo MergeSort(A, esquerda, direita)
    Se esquerda < direita Então
        meio = esquerda + (direita - esquerda) / 2

        # Recursão
        MergeSort(A, esquerda, meio)
        MergeSort(A, meio + 1, direita)

        # Intercalação
        Intercalar(A, esquerda, meio, direita)
    Fim-Se
Fim-Algoritmo

Algoritmo Intercalar(A, esquerda, meio, direita)
    n1 = meio - esquerda + 1
    n2 = direita - meio
```

```
Criar vetores temporários L[0..n1-1] e R[0..n2-1]

Copiar A[esquerda..meio] para L
Copiar A[meio+1..direita] para R

i = 0, j = 0, k = esquerda

Enquanto i < n1 e j < n2 Faça
    Se L[i] <= R[j] Então
      A[k] = L[i]
      i = i + 1
    Senão
      A[k] = R[j]
      j = j + 1
    Fim-Se
    k = k + 1
Fim-Enquanto

# Copiar elementos restantes de L[], se houver
Enquanto i < n1 Faça
    A[k] = L[i]
    i = i + 1
    k = k + 1
Fim-Enquanto

# Copiar elementos restantes de R[], se houver
Enquanto j < n2 Faça
```

```
        A[k] = R[j]
        j = j + 1
        k = k + 1
    Fim-Enquanto
Fim-Algoritmo
```

Complexidade de Tempo e Espaço

- Melhor Caso — O(n log n): A árvore de recursão possui profundidade constante de $\log_2 N$ níveis, e em cada nível a etapa de intercalação consome tempo linear O(n).
- Caso Médio — O(n log n): Análise idêntica à do melhor caso.
- Pior Caso — O(n log n): Garantia determinística de desempenho, sem degradação para distribuições específicas de dados.
- Complexidade de Espaço: O(N) — Requer alocação de vetores auxiliares para a etapa de intercalação (*Out-of-place*).

Estabilidade

Estável. Na função de intercalação, a verificação L[i] <= R[j] garante que, em caso de empate de valores, o elemento vindo do subvetor da esquerda (L) prevaleça e venha primeiro no resultado final.

Aplicação Prática

O Merge Sort é a base dos algoritmos de Ordenação Externa em Banco de Dados (devido ao seu padrão de acesso sequencial a dados) e de estruturas baseadas em Listas Encadeadas (onde a alocação de memória O(N) é dispensada trocando apenas os ponteiros dos nós).

Quick Sort (Ordenação Rápida)

Descrição do Funcionamento

Desenvolvido por Sir C. A. R. Hoare em 1959, o Quick Sort também aplica o paradigma de Divisão e Conquista. Diferente do Merge Sort (que faz a divisão de forma simples e concentra o trabalho na combinação), o Quick Sort executa o trabalho pesado na etapa de particionamento de um conjunto de elementos em relação a um pivô, aplicando recursivamente o processo às partições resultantes (Hoare, 1962).

O processo consiste em:

1. Escolher um elemento do vetor como pivô.
2. Reorganizar o vetor de forma que todos os elementos menores que o pivô fique à sua esquerda, e todos os elementos maiores fiquem à sua direita (particionamento).
3. Aplicar recursivamente o mesmo processo aos subvetores à esquerda e à direita do pivô.

Pseudocódigo (Esquema de Particionamento de Lomuto)

```
Algoritmo QuickSort(A, baixo, alto)
   Se baixo < alto Então
      # p_index é o índice de particionamento, A[p_index] está no lugar correto
      p_index = Particionar(A, baixo, alto)

      QuickSort(A, baixo, p_index - 1)
      QuickSort(A, p_index + 1, alto)
   Fim-Se
Fim-Algoritmo

Algoritmo Particionar(A, baixo, alto)
   pivo = A[alto] # Escolha do último elemento como pivô
   i = baixo - 1

   Para j de baixo até alto - 1 Faça
```

```
      Se A[j] <= pivo Então
        i = i + 1
        trocar(A[i], A[j])
      Fim-Se
   Fim-Para

   trocar(A[i + 1], A[alto])
   Retornar (i + 1)
Fim-Algoritmo
```

Complexidade de Tempo e Espaço

- Melhor Caso — O(n log n): Ocorre quando a etapa de particionamento divide o vetor sempre em duas metades aproximadamente iguais (n/2).
- Caso Médio — O(n log n): Ocorre na vasta maioria das instâncias aleatórias. A constante oculta do Quick Sort na notação Big-O é excepcionalmente pequena, tornando-o o algoritmo de ordenação comparativa em memória mais rápido na prática.

- Pior Caso — O(n^2): Ocorre quando o particionamento produz divisões muito desequilibradas, fazendo com que uma das partições concentre quase todos os elementos e a outra contenha poucos ou nenhum elemento. Embora a implementação utilize a estratégia da mediana de três para selecionar o pivô, essa técnica apenas reduz a probabilidade de partições muito desequilibradas, não eliminando a possibilidade de ocorrência do pior caso.

- Complexidade de Espaço: O(log n) — Devido à pilha de chamadas recursivas no caso médio (pode degradar para O(N) no pior caso se não for otimizado com recursão de cauda).

Estabilidade

Instável. A troca de elementos durante a fase de particionamento não preserva a ordem relativa original entre chaves equivalentes.

Aplicação Prática

O Quick Sort é utilizado em funções padrão de bibliotecas de linguagens como C (qsort), C++ (std::sort, combinada com Heapsort e Insertionsort no *Introsort*), e em mecanismos de otimização de consultas que operam inteiramente em RAM.

Tabela Comparativa Estruturada

Algoritmo	Complexidade (Melhor caso)	Complexidade (Caso Médio)	Complexidade (Pior Caso)	Complexidade Espaço auxiliar	Estável
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Não
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Sim
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	Não

METODOLOGIA

O desenvolvimento deste trabalho consistiu na criação de uma aplicação *web* interativa com o objetivo de realizar uma análise comparativa do desempenho de quatro algoritmos de ordenação (*Insertion Sort*, *Selection Sort*, *Merge Sort* e *Quick Sort*). O sistema foi concebido para processar um *dataset* real contendo registros de focos de queimadas no Brasil entre os anos de 2023 e 2024. A metodologia adotada englobou desde a seleção das tecnologias e arquitetura do sistema até a instrumentação algorítmica para coleta de métricas, subdividindo-se nas etapas descritas a seguir.

Arquitetura do Sistema e Tecnologias

A aplicação foi estruturada em camadas, com separação clara de responsabilidades entre apresentação, lógica de negócio e acesso a dados. O desenvolvimento partiu do padrão arquitetural MVC (*Model-View-Controller*), com renderização de páginas no servidor via Thymeleaf. Ao longo do projeto, porém, a arquitetura evoluiu para uma API REST consumida por um cliente web, de modo a evitar o recarregamento completo da página a cada interação do usuário (detalhado na seção “Arquitetura de comunicação entre *frontend* e *backend*”).

Para o *backend*, adotou-se a linguagem Java com o *framework* Spring Boot, escolhido pela robustez no tratamento de estruturas de dados em memória e pela facilidade de configuração de rotas *web*. Foram utilizados os módulos Spring Web, para a construção dos controladores e dos *endpoints* REST, Thymeleaf, para servir a página inicial (index.html), e Spring Boot DevTools, para agilizar o desenvolvimento por meio de recarregamentos automáticos (*hot reloads*). A estrutura final do *backend* divide-se em:

- Controladores (ApiController e ThymeleafController): recebem as requisições HTTP, validam os parâmetros e devolvem as respostas. O primeiro expõe os *endpoints* REST em JSON; o segundo apenas entrega o index.html.
- Serviços (CsvReaderService, OrdenacaoService e PrevisaoService): concentram as regras de negócio, isto é, a leitura dos arquivos CSV, a execução e a cronometragem dos algoritmos de ordenação e o cálculo da previsão.
- Modelo e DTOs (Registro, ResultadoBenchmark e ResultadoPrevisao): representam os dados do domínio e os objetos transferidos ao cliente.

A interface (*view*) deixou de ser gerada pelo servidor: é uma página estática em HTML, CSS e JavaScript que consome a API por requisições assíncronas.

Modelagem e Preparação dos Dados

O objeto de análise desta pesquisa consistiu em um conjunto de dados do mundo real (registros de focos de queimadas no Brasil no biênio 2023-2024). Os dados foram mapeados no *backend* Java por meio de uma classe de domínio específica (Model), representando cada registro com os seguintes atributos: identificador no banco de dados (id_bdq), identificador do foco (foco_id), latitude (lat), longitude (lon), data de passagem do satélite (data_pas), país (pais), estado (estado), município (municipio) e bioma (bioma).

Devido ao considerável volume de dados, adotou-se uma estratégia de organização e segmentação no nível do sistema de arquivos. Os dados são organizados em diretórios identificados pelas respectivas Unidades Federativas (UF), contemplando os estados brasileiros e o Distrito Federal. Cada diretório contém os arquivos CSV correspondentes aos dados utilizados pelo sistema, permitindo a leitura e o processamento individual ou conjunto dos registros de cada localidade.

O fluxo de processamento de dados baseou-se em estágios sucessivos:

- Filtro Primário: O clique do usuário em um estado específico no mapa SVG aciona o carregamento em memória da sub-base de dados correspondente (pasta específica).
- Critérios de Ordenação: A interface provê caixas de seleção (município, data e bioma) que definem os critérios de ordenação, usados de forma isolada ou combinada. Os critérios marcados são enviados ao *backend*, que monta dinamicamente o comparador de registros (objeto *Comparator*); quando há mais de um critério, eles são encadeados na ordem de seleção e o seguinte atua como critério de desempate. O sistema não filtra registros por município, intervalo de datas ou bioma: o único recorte da base é o estado escolhido no mapa.
- Execução da Ordenação: A etapa final do fluxo aplica o comparador montado na etapa anterior, sempre em ordem ascendente, ao conjunto de registros do estado selecionado. Esse conjunto forma o *array* de entrada (vetor de teste); cada algoritmo recebe uma cópia idêntica desse vetor e é cronometrado individualmente.

Implementação dos Algoritmos de Ordenação e Instrumentação de Métricas

Para avaliar o desempenho dos algoritmos, foram utilizadas duas abordagens complementares: a contagem de operações elementares e a medição do tempo de execução. A primeira permite observar o esforço computacional realizado por cada algoritmo, enquanto a segunda registra o tempo necessário para concluir o processamento. O tempo foi aferido no backend por meio do método System.nanoTime(), buscando medir o intervalo de execução dos algoritmos sem incluir a latência da comunicação pela rede.

Para realizar a coleta, cada método de ordenação não apenas ordenou o vetor, mas recebeu como parâmetro uma instância de uma classe criada especificamente para registrar a instrumentação. Esta classe funcionou como um acumulador encapsulado, contabilizando as seguintes métricas fundamentais:

- Comparações: Toda expressão condicional envolvendo a checagem da grandeza entre dois objetos do vetor (utilizando métodos análogos ao compareTo da interface Comparable) provocou o incremento de um contador de comparações.
- Escritas (Atribuições/Movimentações): Toda vez que um elemento foi inserido em uma posição do *array*, seja na reatribuição de valores de um algoritmo *in-place* ou na cópia para *arrays* auxiliares (como exigido pela etapa de conquista do *Merge Sort*), contabilizou-se uma operação de escrita (também denominada movimentação de dados).
- Trocas (*Swaps*): Quando ocorreu a permuta explícita de dois elementos dentro do vetor (presente fundamentalmente no *Selection* e na rotina de particionamento do *Quick Sort*), um contador específico registrou a operação, evidenciando o custo associado à permutação posicional.

Cada algoritmo foi executado sequencialmente sobre o *mesmo* vetor inicial gerado no estágio de filtragem (para tanto, garantiu-se que cópias independentes do vetor desordenado fossem entregues a cada algoritmo). Os resultados coletados da classe instrumentadora forneceram métricas consolidadas sobre o comportamento individual de cada estrutura lógica no respectivo cenário de dados.

Design da Interface Web e *Dashboard* de Resultados

Após o processamento realizado no *backend*, o controlador Spring construiu um modelo de resposta contendo os dados ordenados e os atributos coletados referentes à instrumentação dos quatro algoritmos.

O *Dashboard* foi projetado para facilitar a interpretação da análise de desempenho. Os indicadores foram apresentados por meio de *cards* e tabelas informativas comparando quantitativamente as métricas de contagem de comparações, escritas e trocas executadas. Adicionalmente, permitiu-se aos usuários visualizar a estrutura dos dados processados, com seus campos expostos, completando o ciclo interativo e possibilitando uma verificação empírica acerca da distribuição das classes, tipos e volumes do *dataset* após o término de processamento dos algoritmos.

PROJETO DO PROGRAMA

Este capítulo detalha a concepção técnica do software desenvolvido, abordando desde as escolhas arquitetônicas até a implementação prática e validação das estruturas de dados. O foco central reside na aplicação e análise de desempenho de diferentes algoritmos de ordenação sobre uma base de dados massiva, processando registros de focos de incêndio no território brasileiro extraídos do INPE.

Arquitetura de comunicação entre frontend e backend

Inicialmente, o projeto utilizava o padrão MVC com o Thymeleaf renderizando as páginas no servidor. Essa abordagem exigia o recarregamento completo da página a cada interação, o que prejudicava a experiência ao executar vários benchmarks seguidos. Optou-se então por uma API REST: na arquitetura final, o Thymeleaf serve apenas o index.html, e todas as interações (escolha do estado, envio dos critérios de ordenação e exibição dos resultados) ocorrem por requisições HTTP feitas em JavaScript. Manter o *frontend* dentro do mesmo projeto Spring Boot dispensou a criação de dois projetos separados, simplificando a execução e a entrega do trabalho. O *backend* permanece responsável pelas regras de negócio, pela execução dos algoritmos e pela disponibilização dos resultados pelos *endpoints*.

Processamento de ordenação e previsão

O sistema possui dois fluxos distintos de ordenação, cada um com uma finalidade específica.

O primeiro fluxo corresponde à ordenação solicitada pelo usuário. Após a seleção da Unidade Federativa e dos filtros disponíveis, o backend realiza a filtragem dos registros e executa os algoritmos de ordenação conforme os critérios selecionados. Os resultados são utilizados para apresentar os dados organizados na interface e permitir a comparação do desempenho dos algoritmos implementados.

O segundo fluxo ocorre internamente no backend e está associado à funcionalidade de previsão de queimadas. Nesse processo, os registros históricos são organizados cronologicamente, independentemente dos critérios de ordenação selecionados pelo usuário. Essa organização é necessária para que os dados sejam utilizados na sequência temporal correta durante a aplicação dos cálculos matemáticos responsáveis por estimar o comportamento dos focos de queimadas no período seguinte.

Dessa maneira, o sistema mantém separadas a ordenação destinada à apresentação dos resultados e a organização cronológica necessária ao processamento da previsão, permitindo que ambas as funcionalidades operem com finalidades distintas.

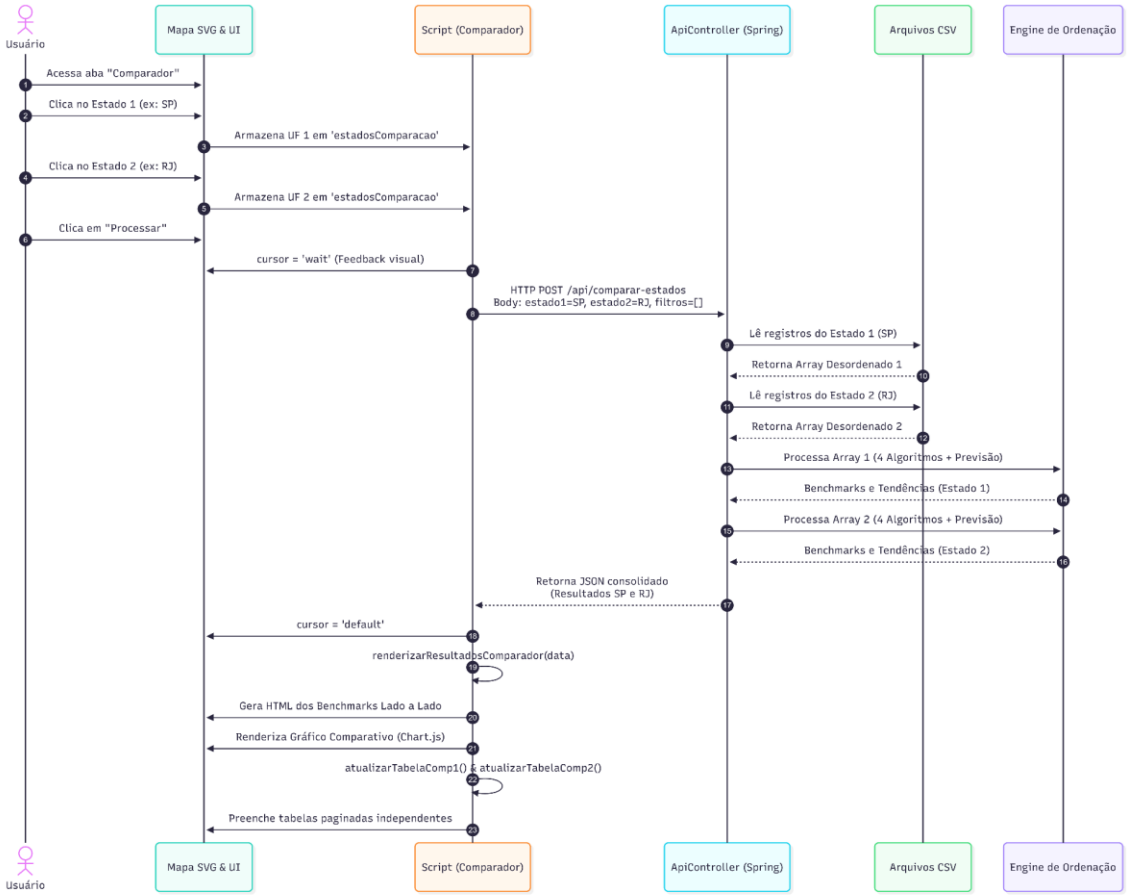


Figura 1 Diagrama do funcionamento geral do projeto

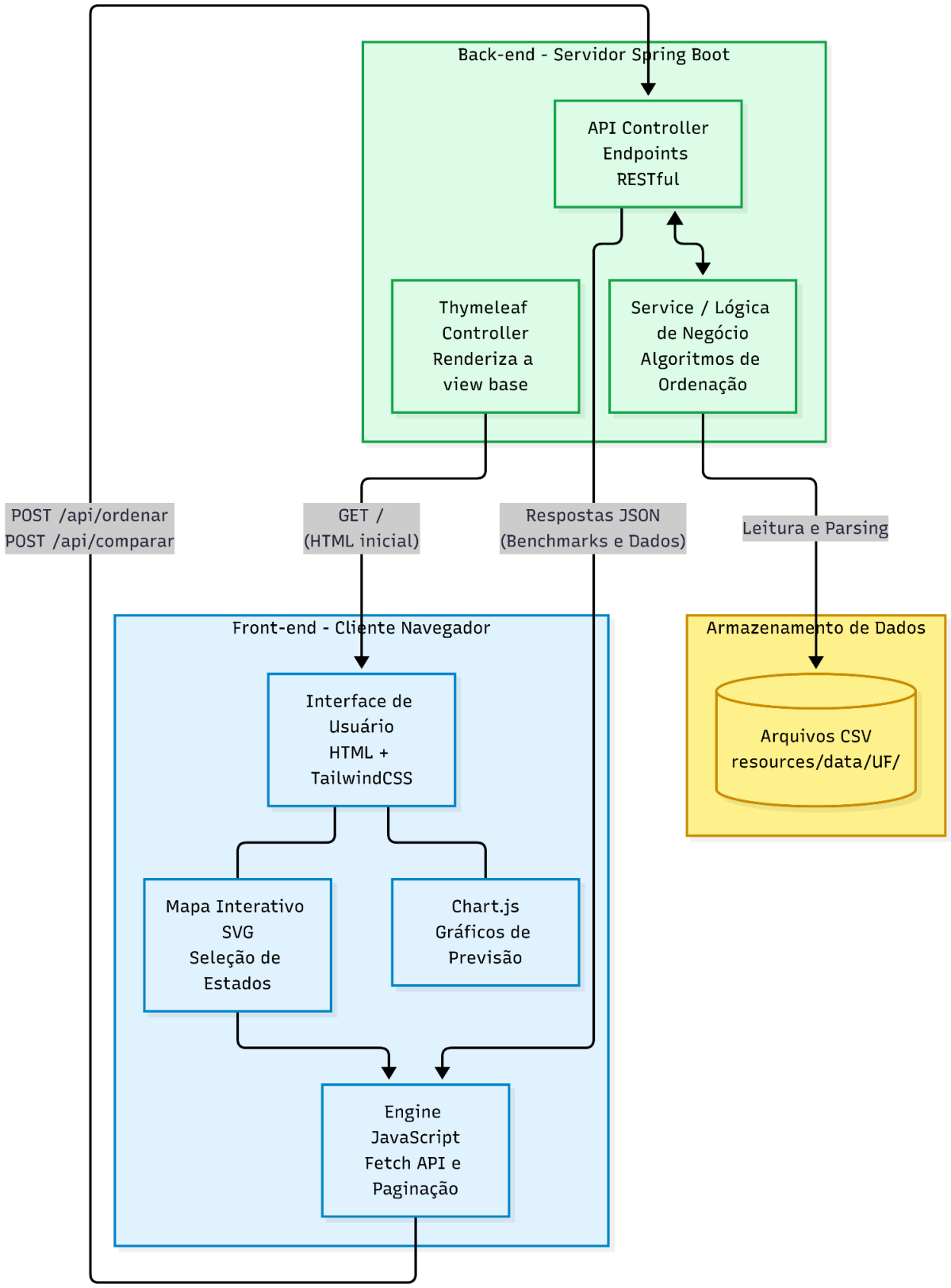


Figura 2 diagrama do funcionamento da API

Demonstração do uso de algoritmos de ordenação

O núcleo do projeto consiste na aplicação de algoritmos de ordenação para organizar os dados históricos de queimadas sob diferentes critérios, operando nos *endpoints* da API. Para o estudo comparativo de complexidade e tempo de execução, foram implementados quatro algoritmos distintos:

1. Insertion Sort (Ordenação por Inserção): Algoritmo de complexidade $O(N^2)$, ideal para volumes reduzidos, mas de alto custo para o dataset do INPE. A matriz é ordenada de forma incremental.
2. Selection Sort (Ordenação por Seleção): Também de complexidade $O(N^2)$, divide a estrutura em uma sublista ordenada e outra desordenada. Foi utilizado para avaliar o impacto do número de trocas de variáveis na memória do servidor durante as chamadas da API.
3. Merge Sort (Ordenação por Intercalação): Utiliza o paradigma "Divisão e Conquista" com complexidade O(n log n). Divide recursivamente a base de dados até atingir unidades únicas, mesclando-as ordenadamente. É crucial para que a API consiga devolver a resposta JSON em tempo hábil para o Front-end ao lidar com grandes recortes do dataset, possuindo complexidade espacial O(n).
4. Quick Sort (Ordenação Rápida): Algoritmo O(n log n) baseado na escolha de um *pivô* para particionar os dados. Por ser uma ordenação *in-place* (complexidade espacial O(log n)), destacou-se pelo baixo consumo de memória auxiliar.

Testes, validações e iterações (Benchmarks)

O processo de engenharia de software foi guiado por testes empíricos de performance, capturando o tempo de execução no nível do back-end (System.nanoTime()) para isolar a latência da rede e medir o processamento real da CPU.

Avaliação de Desempenho e Benchmarks: O sistema foi submetido a testes de carga utilizando recortes sequenciais do dataset. Os testes evidenciaram o abismo de performance entre os algoritmos de complexidade quadrática e os de complexidade O(n log n) quando expostos ao volume massivo de focos de queimada.



Figura 3 Fonte: Autoria própria

341 dados: Com o menor volume de entrada, todos os algoritmos apresentam tempos muito curtos, com o Quick Sort liderando (0,1 ms), seguido de perto pelo Merge Sort (0,5 ms), Insertion Sort (0,7 ms) e Selection Sort (1 ms).



Figura 4 Fonte: Autoria própria

1203 dados: O Merge Sort assume a liderança absoluta com apenas 0,2 ms, enquanto o Quick Sort registra 0,7 ms. O Selection e o Insertion começam a demonstrar o crescimento de sua complexidade, marcando 6,8 ms e 4,8 ms, respectivamente.



Figura 5 Fonte: Autoria própria

2025 dados: A divisão entre algoritmos O(n log n) e quadráticos torna-se visível; Merge (0,4 ms) e Quick Sort (1,2 ms) mantêm excelente desempenho. Entre os mais lentos, ocorre uma inversão pontual, com o Selection (25,2 ms) executando levemente mais rápido que o Insertion (26,9 ms).



Figura 6 Fonte: Autoria própria

4143 dados: A curva de tempo do Selection e Insertion sofre uma aceleração drástica, atingindo 135,2 ms e 64,81 ms. Em total contraste, a base do gráfico permanece estável com o Merge (1,5 ms) e Quick Sort (3,7 ms).



Figura 7 Fonte: Autoria própria

8017 dados: O Selection (464,9 ms) torna-se mais de duas vezes mais lento que o Insertion (222,74 ms). O Quick Sort apresenta uma elevação sensível para 44,6 ms, mas o Merge Sort permanece extremamente eficiente com apenas 1,8 ms.



Figura 8 Fonte: Autoria própria

15220 dados: O abismo no desempenho se alarga substancialmente; o Selection ultrapassa a marca de dois segundos (2003,1 ms) e o Insertion chega a 865,72 ms. O Quick Sort salta para 229,5 ms, enquanto o Merge Sort segue quase inalterado em 5,2 ms.



Figura 9 Fonte: Autoria própria

21077 dados: Observa-se uma variação de performance interessante no Quick Sort, que cai para 78,4 ms em relação à linha anterior. O Merge Sort segue sua trajetória de crescimento suave, compatível com O(n log n), marcando 7,6 ms. O Selection (3464,7 ms) e o Insertion (2037,7 ms) continuam em crescimento quadrático.



Figura 10 Fonte: Autoria própria

43992 dados: O Selection alcança 7073,3 ms e o Insertion 6300,5 ms, reduzindo a proporção de diferença de tempo entre eles. O Merge Sort (35,7 ms) e o Quick Sort (227,8 ms) mantêm a superioridade inerente à sua arquitetura.



Figura 11 Fonte: Autoria própria

97785 dados: No extremo do gráfico, a ineficiência para grandes volumes é evidente: o Selection leva 42141,2 ms (mais de 42 segundos) e o Insertion 27489,08 ms. O Quick Sort finaliza em 2059,09 ms, e o Merge Sort consolida sua dominância absoluta com impressionantes 87,3 ms.

Ambiente e procedimentos de execução dos testes

Os experimentos foram realizados em um computador equipado com processador AMD Ryzen 5 5600X, 16 GB de memória RAM e sistema operacional Windows 11.

Foram utilizadas diferentes quantidades de registros, partindo de 341 registros até 97.785 registros. Para cada quantidade de dados, os quatro algoritmos foram submetidos ao processamento utilizando exatamente o mesmo conjunto de registros, garantindo condições equivalentes quanto à entrada utilizada na comparação.

Durante cada execução, foram registrados o tempo de processamento e as métricas de operações contabilizadas pela aplicação. Cada combinação de algoritmo e quantidade de registros foi executada uma única vez. Portanto, os tempos apresentados correspondem aos resultados observados individualmente, não representando médias estatísticas de múltiplas repetições.

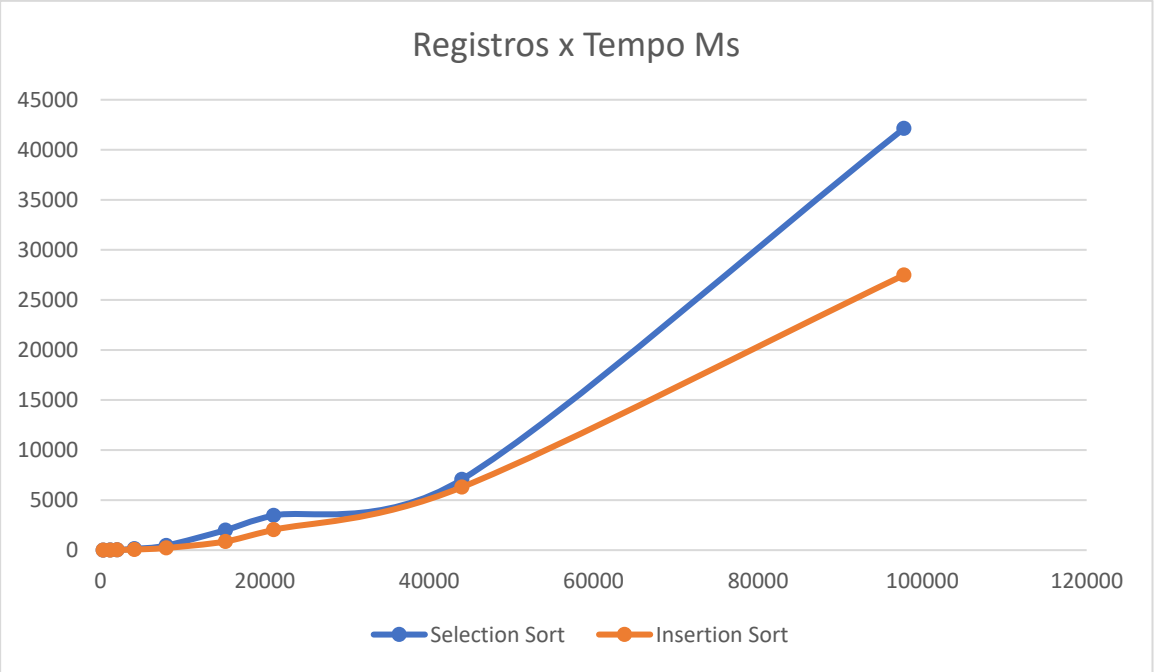
A utilização dos mesmos registros permite uma comparação mais controlada entre os algoritmos, pois reduz a influência de diferenças na entrada sobre os resultados. Entretanto, a ausência de execuções repetidas limita a avaliação da variabilidade dos tempos, que também podem ser influenciados por fatores externos, como o funcionamento do sistema operacional e o gerenciamento de memória durante o processamento.

Dessa forma, os resultados permitem observar o comportamento dos algoritmos nas condições experimentais utilizadas, mas não devem ser interpretados como uma medição estatística definitiva de seu desempenho.

Análise Comparativa: Selection Sort x Insertion Sort

Para avaliar o comportamento dos algoritmos de complexidade quadrática $O(n^2)$, foram realizados testes incrementais com lotes de dados variando de 341 a 97.785 registros de queimadas.

- Comportamento em pequenas escalas: Em volumes muito baixos, como 341 registros, ambos apresentam tempos quase imperceptíveis (1 ms para Selection e 0,7 ms para Insertion).
- Curva de degradação: À medida que a carga aumenta, o crescimento quadrático de $O(n^2)$ se torna evidente. Ao atingir 43.992 registros, o Selection Sort saltou para 7.073,3 ms (mais de 7 segundos), enquanto o Insertion Sort marcou 6.300,5 ms.
- Estresse máximo do teste: No ápice do benchmark (97.785 registros), o tempo de execução sofreu uma explosão computacional. O Selection Sort cravou 42.141,2 ms (aproximadamente 42 segundos), ao passo que o Insertion Sort finalizou em 27.489,08 ms (cerca de 27 segundos).



Conclusão da comparação: Embora ambos possuam a mesma notação Big-O, o Insertion Sort demonstrou ser significativamente mais eficiente (cerca de 35% mais rápido no pior caso testado). Isso ocorre porque o Insertion Sort consegue interromper as comparações internas assim que encontra a posição correta do elemento (sendo altamente adaptável a dados parcialmente ordenados), enquanto o Selection Sort obriga a CPU a varrer o array restante inteiro em todas as iterações, independentemente do estado dos dados. Contudo, tempos de 27 a 42 segundos são inadequados para uma API REST interativa, pois o usuário aguardaria dezenas de segundos pela resposta, com risco de *timeout* na requisição do front-end.

Análise Comparativa: Merge Sort x Quick Sort

A avaliação dos algoritmos de paradigma "Divisão e Conquista", que operam com complexidade média de $O(n \log n)$, revelou comportamentos muito distintos ao processar os dados reais do INPE.

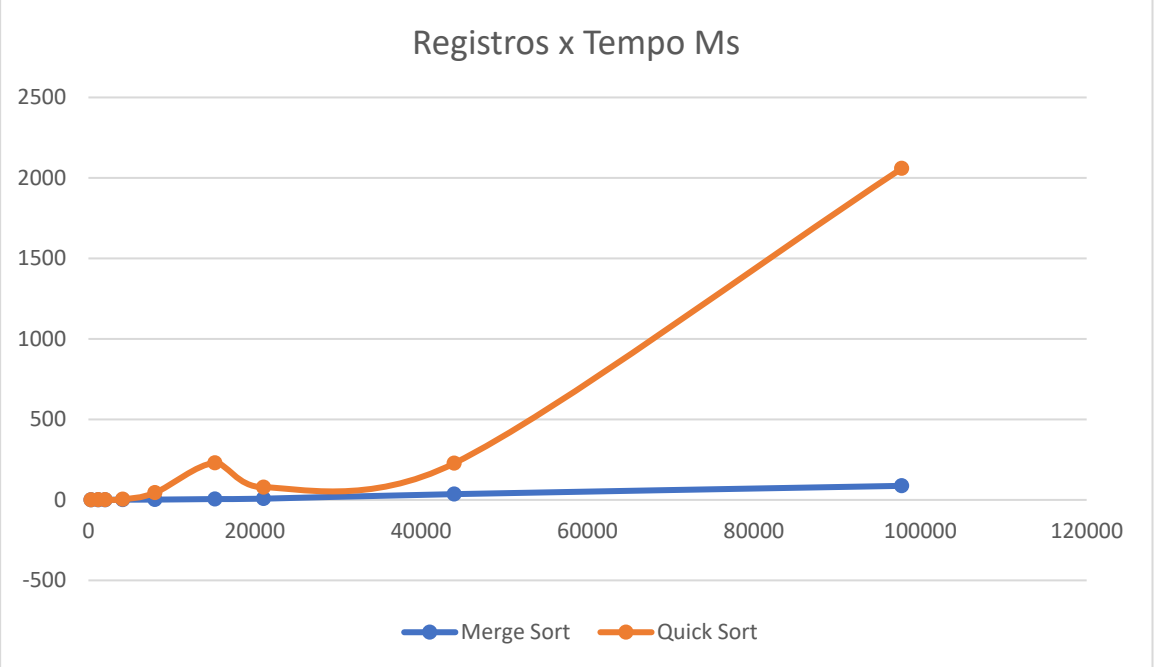
- Desempenho em alta escala: Para a carga máxima de 97.785 registros, o Merge Sort entregou o resultado em impressionantes 87,3 ms. Em contrapartida, o Quick Sort apresentou uma latência muito maior nesse mesmo cenário, levando 2.059,09 ms (mais de 2 segundos) para concluir a ordenação.

- Análise da variação de desempenho do Quick Sort: Durante os testes, observou-se uma variação nos tempos de execução do Quick Sort conforme a quantidade de registros processados. Um exemplo ocorreu com 15.220 registros, cuja execução levou 229,5 ms, enquanto o conjunto de 21.077 registros foi processado em 78,4 ms.

Essa diferença demonstra que o tempo de execução não depende exclusivamente da quantidade de elementos, podendo também ser influenciado pela distribuição e pela ordem inicial dos registros, bem como pelo comportamento das partições geradas durante a execução.

A implementação pretende usar a mediana de três para escolher o pivô, mas, na prática, a rotina deixa o maior dos três elementos como pivô. Somado ao particionamento com <=, que concentra elementos iguais ao pivô em um único lado, isso desbalanceia as partições quando há muitas chaves repetidas. Como o experimento não registrou o critério de ordenação nem o tamanho das partições, essa explicação é uma hipótese a ser confirmada.

- Desempenho do Merge Sort: O Merge Sort apresentou os menores tempos de execução nos testes com maiores volumes de registros. No conjunto de 97.785 elementos, o algoritmo concluiu a ordenação em 87,3 ms. Esse comportamento está de acordo com sua complexidade teórica de (O(n \log n)), que apresenta crescimento assintótico inferior ao dos algoritmos quadráticos avaliados.



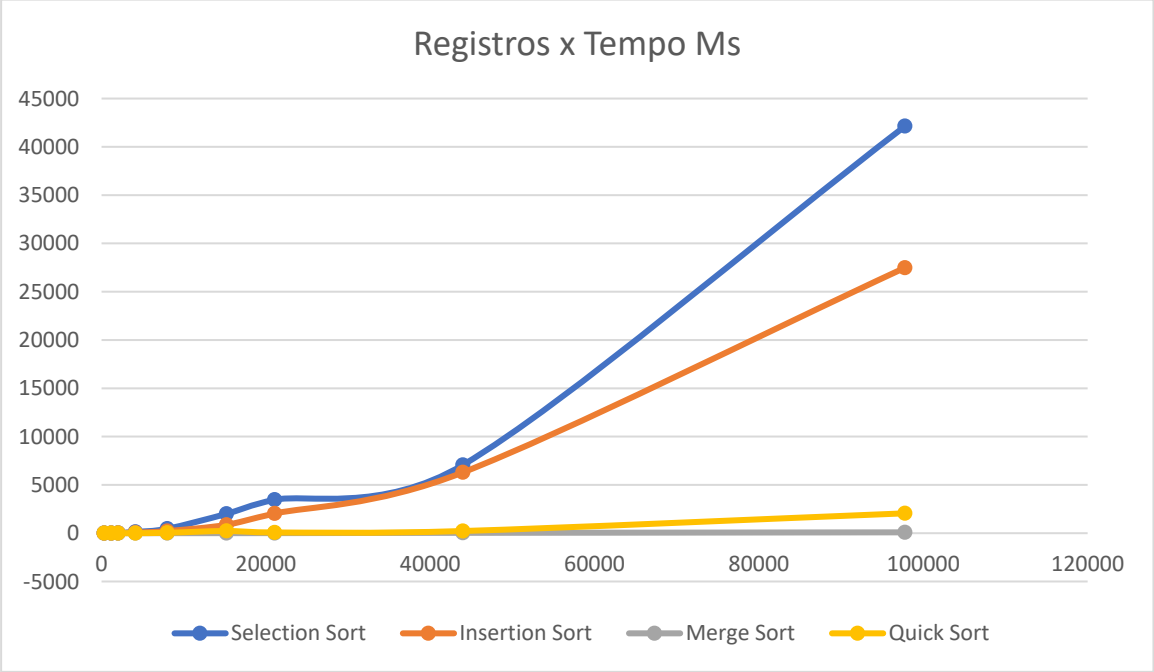
Comparativo Geral Consolidado

Ao colocar os quatro algoritmos lado a lado no gráfico geral, a diferença de desempenho entre as abordagens torna-se evidente .

Ordenar quase 100 mil focos de incêndio custou 42.141,2 milissegundos utilizando o Selection Sort, contra meros 87 milissegundos utilizando o Merge Sort. O Merge Sort (O(n log n)) foi aproximadamente 480 vezes mais rápido que o Selection Sort (quadrático).

No contexto do software desenvolvido (uma API Spring Boot consumida assincronamente por um mapa SVG no navegador), o uso de algoritmos $O(n^2)$ provou-se destrutivo. Requisições que duram dezenas de segundos bloqueiam as *threads* do servidor Tomcat, consomem recursos excessivos da JVM e destroem a experiência do usuário.

Por outro lado, o Merge Sort apresentou crescimento previsível no tempo de execução, mesmo com o aumento da quantidade de registros, respondendo ao cliente em frações de segundo. Esses resultados indicam que o Merge Sort é a opção mais adequada para uma eventual versão de produção; nesta aplicação, os quatro algoritmos são executados a cada requisição com finalidade comparativa.

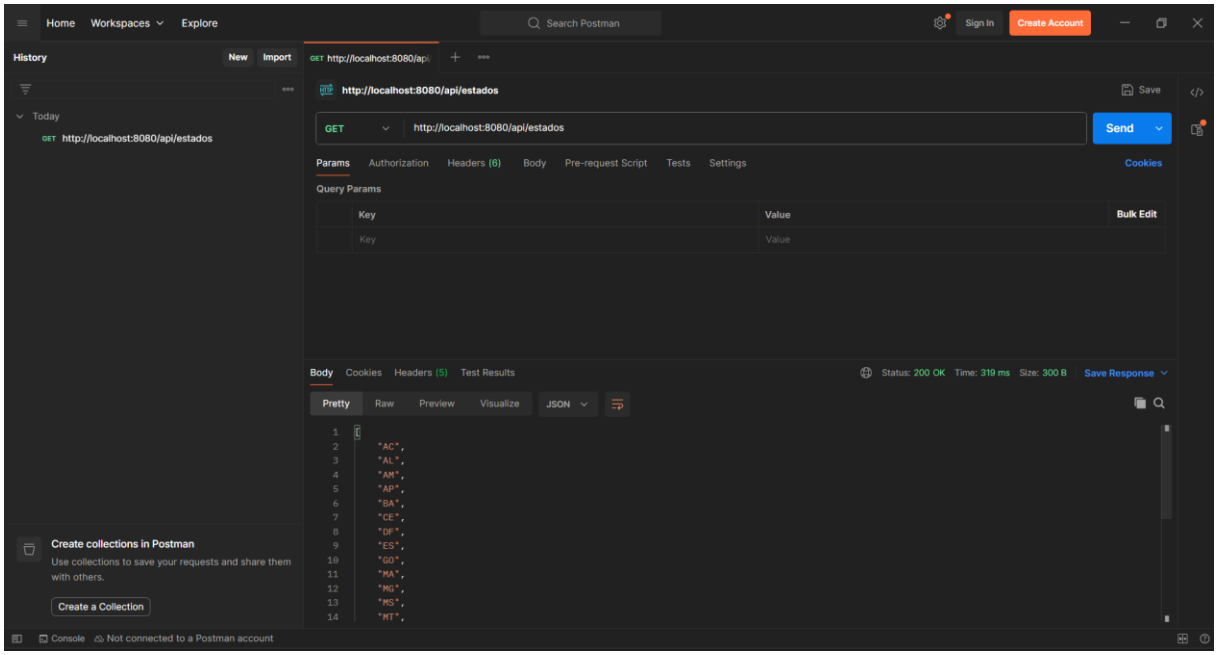


Testes dos Endpoints da API (ApiController)

Com a refatoração para uma arquitetura desacoplada, a comunicação entre o backend (Spring Boot) e os clientes passa a ser realizada exclusivamente via requisições HTTP REST. Abaixo encontram-se descritos os endpoints implementados na classe *ApiController*, juntamente com os cenários de testes e validações realizados via Postman.

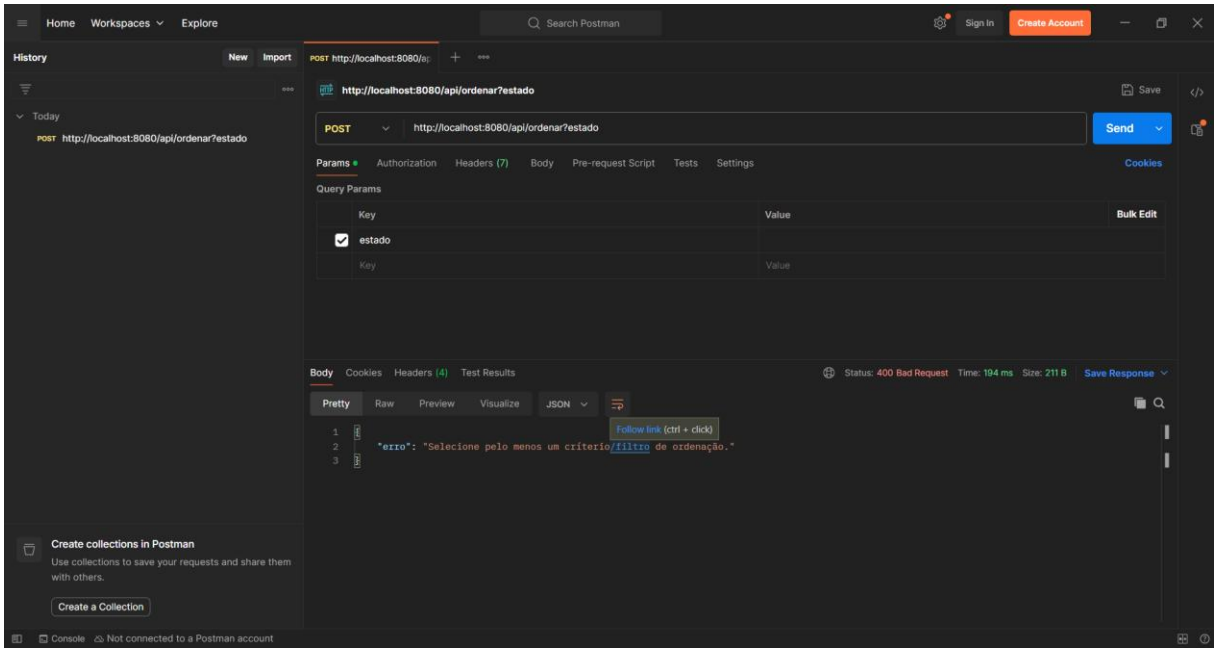
Listar Estados Disponíveis

- Método e Rota: *GET /api/estados*
- Descrição: Retorna a coleção completa de siglas de estados disponíveis na base de dados do INPE para seleção nas interfaces.
- Resultado do Teste: Retorno *200 OK* contendo um array JSON estruturado com as siglas correspondentes.

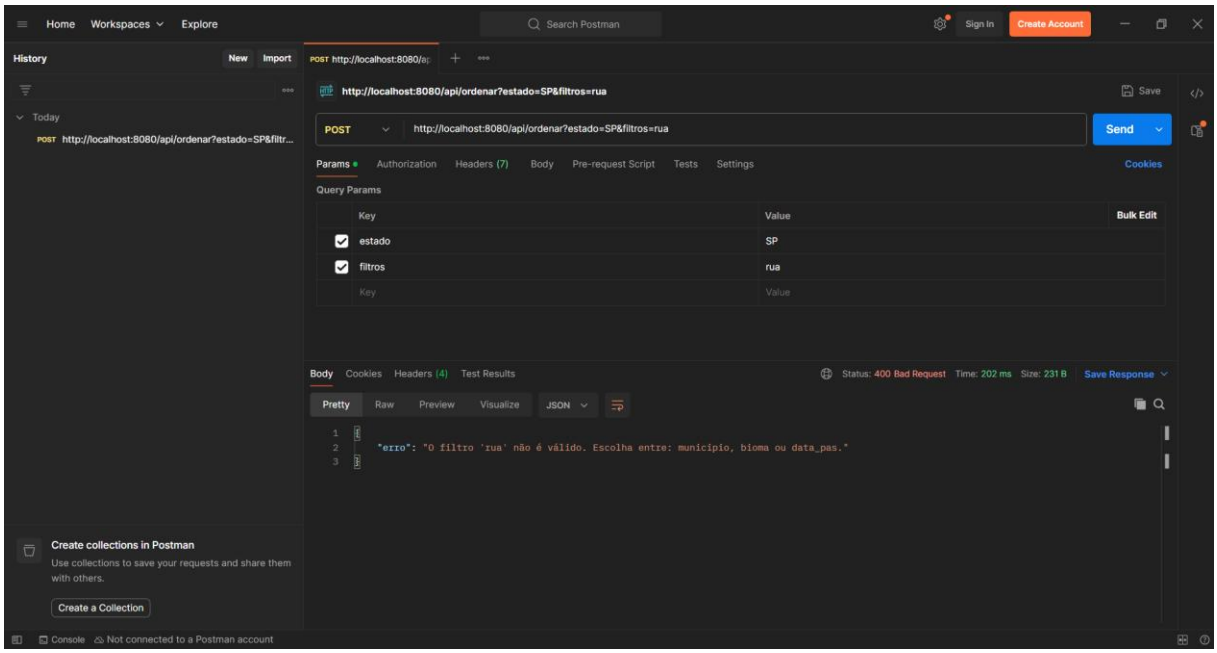


Processamento de Benchmark e Ordenação (1 Estado)

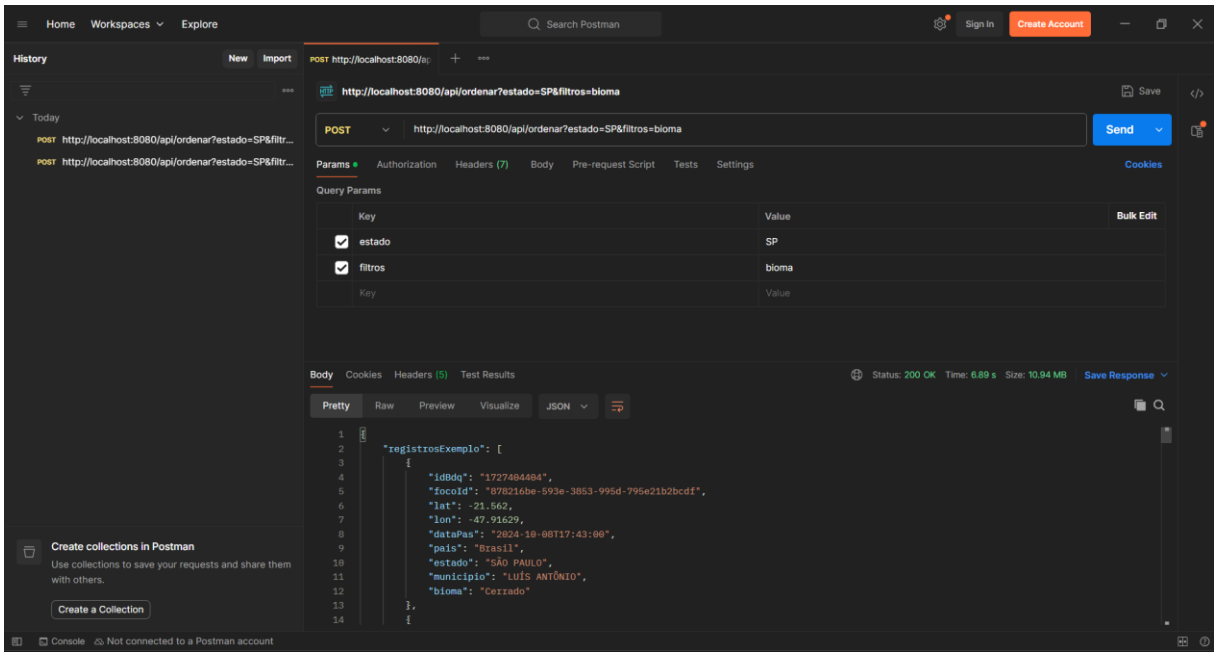
- Método e Rota: *POST /api/ordenar*
- Parâmetros Exigidos: *estado* (String) e *filtros* (List).
- Cenários de Validação e Erros Tratados:
    - Filtros Vazios / Ausentes: Retorna código *400 Bad Request* com a mensagem *"Selecione pelo menos um critério/filtro de ordenação."*.



- Filtro Inválido: Caso o utilizador envie parâmetros não mapeados (ex: *rua*), a API valida os critérios permitidos e retorna um *400 Bad Request* (*"O filtro 'rua' não é válido..."*), prevenindo exceções internas no servidor.

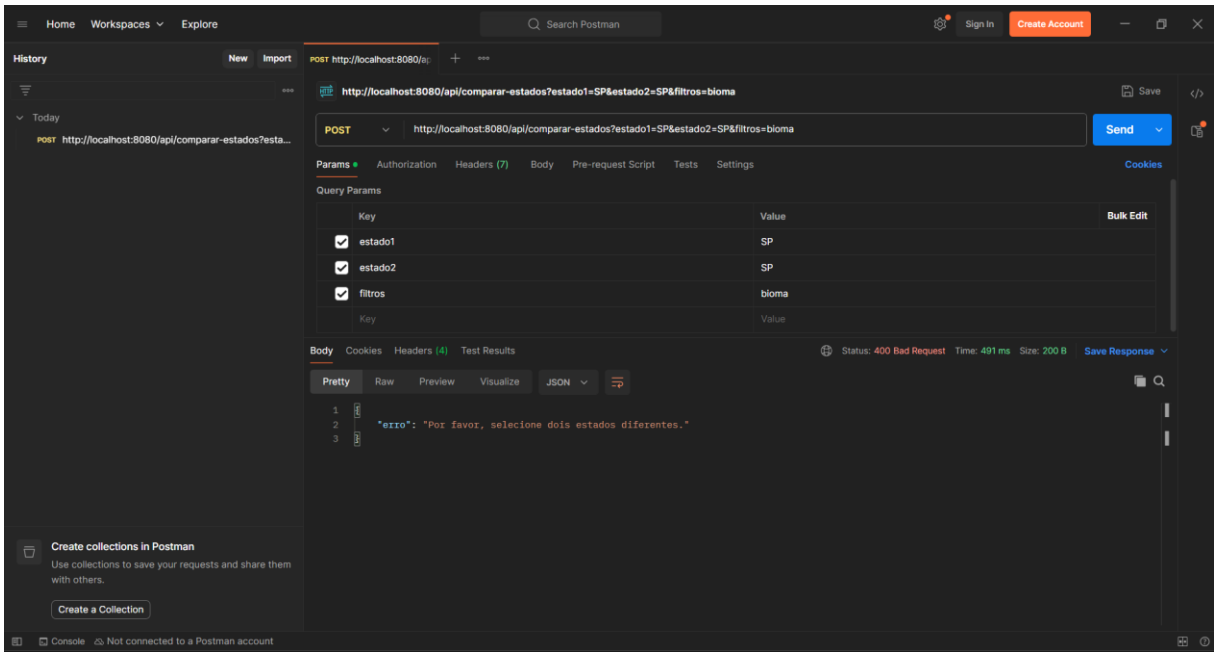


- Sucesso (*200 OK*): Devolve o benchmark completo de desempenho dos algoritmos, amostras ordenadas e a previsão baseada nos dados cronológicos.

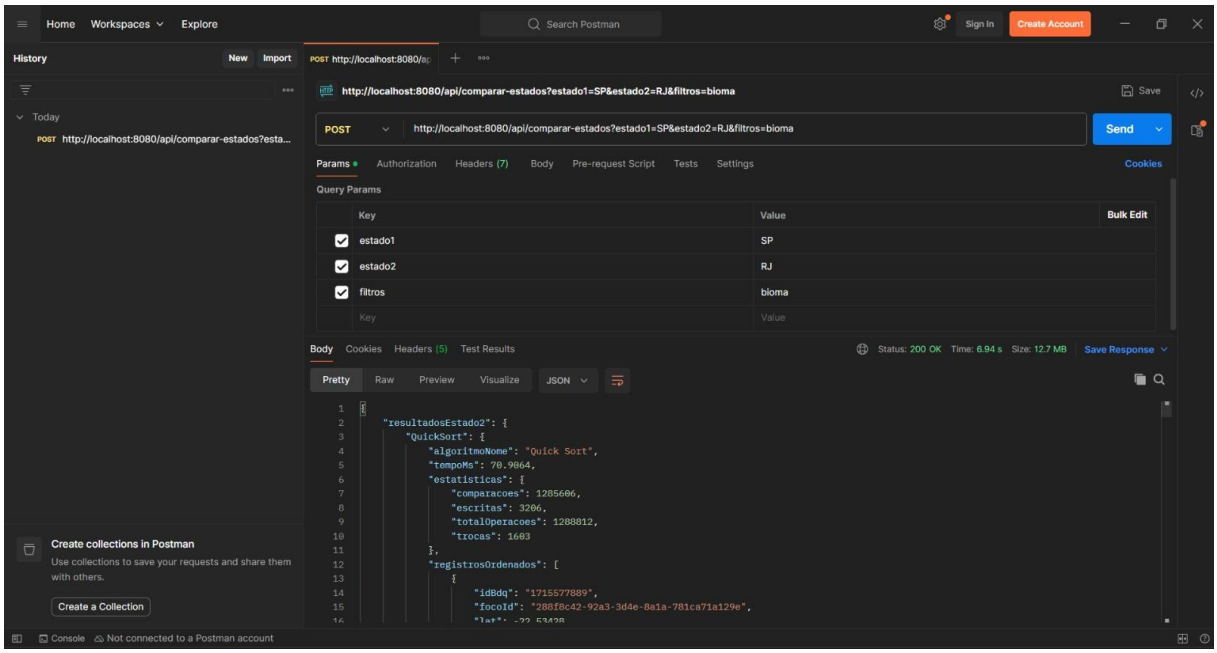


Comparativo entre Estados

- Método e Rota: *POST /api/comparar-estados*
- Parâmetros Exigidos: *estado1* (String), *estado2* (String) e filtros (List).
- Cenários de Validação e Erros Tratados:
  - Estados Iguais: Valida se *estado1* é igual a *estado2*, retornando *400 Bad Request* com o aviso *"Por favor, selecione dois estados diferentes."*.



- Sucesso (*200 OK*): Retorna um objeto JSON consolidando os resultados comparativos de benchmark e previsões para ambos os estados em simultâneo.



CONCLUSÃO

A realização deste projeto permitiu confrontar a análise teórica de complexidade com o comportamento medido de quatro algoritmos de ordenação (*Insertion Sort*, *Selection Sort*, *Merge Sort* e *Quick Sort*), implementados em Java e aplicados a registros reais de focos de queimadas do INPE (2023 e 2024). Foram atendidos os objetivos propostos: importação dos arquivos CSV com descarte de linhas inconsistentes, ordenação dos registros de um estado por município, data e bioma (isolados ou combinados), registro de comparações, escritas, trocas e tempo de execução de cada algoritmo, e apresentação dos resultados em um painel web com mapa interativo. O sistema ainda incorporou um comparador entre dois estados e uma previsão do mês seguinte, obtida por regressão linear simples sobre a contagem mensal de focos.

Os testes confirmaram a diferença entre as classes de complexidade. Os algoritmos quadráticos mostraram crescimento acentuado: para 97.785 registros, o *Selection Sort* levou 42.141,2 ms e o *Insertion Sort* 27.489,08 ms, cerca de 35% menos, pois o segundo interrompe o laço interno assim que encontra a posição correta, enquanto o primeiro percorre todo o restante do vetor a cada iteração. Tempos dessa ordem são inadequados para uma interface interativa. Como o sistema é uma ferramenta de comparação, ele executa os quatro algoritmos a cada requisição, e por isso a resposta para estados grandes é limitada pelos algoritmos quadráticos. Em uma aplicação real, apenas algoritmos O(n log n) seriam empregados.

O *Merge Sort* foi o algoritmo mais regular: 5,2 ms com 15.220 registros, 7,6 ms com 21.077, 35,7 ms com 43.992 e 87,3 ms com 97.785. Esse comportamento é compatível com sua complexidade O(n log n), que independe da disposição inicial dos dados, e com sua estabilidade. Entre os algoritmos avaliados, é o mais indicado para ordenar esse conjunto de dados.

O *Quick Sort*, embora tenha a mesma complexidade média, foi bem mais lento e irregular: 2.059,09 ms com 97.785 registros, contra 87,3 ms do *Merge Sort*, e 229,5 ms com 15.220 registros contra 78,4 ms com 21.077. Isso mostra que seu tempo depende do equilíbrio das partições, e não apenas da quantidade de elementos. A revisão da implementação indica duas causas prováveis. A primeira é que a rotina de escolha do pivô, que pretende usar a mediana de três, deixa na prática o maior dos três elementos como pivô. A segunda é que o particionamento com comparação <= coloca todos os elementos iguais ao pivô do mesmo lado, e os dados de queimadas têm muitas chaves repetidas (por exemplo, todos os registros de um estado pertencem ao mesmo bioma). Nesses casos as partições ficam desbalanceadas e o custo se aproxima de $O(n^2)$. Como os experimentos não registraram o critério de ordenação de cada teste nem o tamanho das partições, essa explicação permanece como hipótese a ser confirmada. Ela indica que a

lentidão decorre dos detalhes da implementação e dos dados, e não do *Quick Sort* em si.

Os resultados têm limitações. Cada combinação de estado e algoritmo foi medida uma única vez, em uma só máquina e sem aquecimento da JVM, portanto os tempos são indicativos e não médias estatísticas. Os arquivos CSV são lidos do disco a cada requisição, e o único recorte de dados disponível é o estado: município, data e bioma são usados apenas como critérios de ordenação. A previsão é uma regressão linear sobre apenas 24 meses e deve ser lida como ilustração da utilidade da organização temporal dos dados, não como modelo preditivo validado.

Como trabalhos futuros, propõem-se: corrigir a seleção do pivô e adotar particionamento de três vias para tratar chaves repetidas, repetindo os testes do *Quick Sort*; realizar múltiplas execuções por cenário, com média e desvio-padrão; implementar filtros reais por município, intervalo de datas e bioma; adicionar cache da leitura dos CSVs; criar testes unitários que verifiquem a correção da ordenação; e incluir a ordenação nativa do Java (List.sort, baseada em Timsort) como referência de comparação.

Conclui-se que a escolha do algoritmo de ordenação teve impacto direto e mensurável sobre o tempo de resposta, e que a implementação e a natureza dos dados importam tanto quanto a notação Big-O. A ordenação eficiente mostrou-se indispensável para transformar arquivos brutos em análises geográficas ágeis e contribui para sistemas de monitoramento ambiental mais responsivos.

BIBLIOGRAFIA

- KNUTH, Donald Ervin. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2. ed. Reading, MA: Addison-Wesley, 1998.
- CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.
- HOARE, Charles Antony Richard. Quicksort. *The Computer Journal*, v. 5, n. 1, p. 10-16, 1962.
- INSTITUTO NACIONAL DE PESQUISAS ESPACIAIS (INPE). Programa Queimadas: Banco de Dados de Queimadas (BDQueimadas). Disponível em: https://terrabrasilis.dpi.inpe.br/queimadas/bdqueimadas/. Acesso em: 3 nov. 2026.
- INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA (IBGE). Biomas. Rio de Janeiro: IBGE. Disponível em: https://www.ibge.gov.br/geociencias/cartas-e-mapas/informacoes-ambientais/15842-biomas.html. Acesso em: 3 nov. 2026.

ANEXOS

Código GitHub:
https://github.com/J040Fr0n3/APS---Algoritmos-de-ordenacao---UNIP-2-2026

Youtube:

Fichas Técnicas: